

UNIVERSITÀ DEGLI STUDI DI SALERNO

**DIPARTIMENTO DI INGEGNERIA
DELL'INFORMAZIONE E DELLA MATEMATICA
APPLICATA**



CORSO DI LAUREA TRIENNALE

**“Sviluppo di un Agente di Guida Autonoma tramite
Imitation Learning per la IBM AI Race League”**

DOCENTI:

Prof. Alessia Saggese

Prof. Mario Vento

STUDENTI Gruppo 20:

Daniele Capaldo - 0612709881

Matteo Muccio - 0612709614

Nicolò Massimo Lisena - 0612708994

Myriam Francesca Spinelli - 0612800678

ANNO ACCADEMICO 2025/2026

Descrizione del progetto

Il presente documento illustra le attività di ricerca e sviluppo condotte dal team DYNEM (Università di Salerno) nell'ambito della IBM AI Race League, una competizione che valuta l'efficienza degli agenti autonomi in pista, unitamente alla qualità dell'architettura software e della relativa documentazione tecnica.

L'obiettivo primario del progetto consiste nello sviluppo di un agente di guida autonoma in grado di minimizzare i tempi di percorrenza sul circuito "Corkscrew" all'interno del simulatore TORCS (The Open Racing Car Simulator). A tale scopo, è stato adottato un approccio basato sull'Imitation Learning, e più specificamente sul Behavioral Cloning, assistito dall'utilizzo del modello IBM Granite come assistente per la stesura del codice. Tale approccio è stato preferito al Reinforcement Learning per via della maggiore stabilità algoritmica e dell'assenza di lunghi periodi di esplorazione.

Il report analizza l'intera pipeline di sviluppo, strutturata nelle seguenti fasi:

- **Acquisizione Dati:** Creazione di un dataset comportamentale tramite la registrazione dei dati e degli input di controllo durante sessioni di guida manuale.
- **Architettura Software:** Progettazione di un sistema modulare, scalabile ed efficiente per la gestione delle comunicazioni UDP con il simulatore e per l'elaborazione dei flussi di dati.
- **Addestramento del Modello:** Implementazione di un modello di regressione multi-output basato sull'algoritmo K-Nearest Neighbors (KNN Regressor) per mappare lo spazio degli stati vettura nelle corrispondenti azioni di controllo continue.

Architettura del sistema

Modellando direttamente una funzione di mappatura tra gli stati del veicolo e le azioni ottimali del pilota, il Behavioral Cloning minimizza i comportamenti anomali in fase di training, garantendo una fedele riproduzione delle traiettorie e delle dinamiche di accelerazione, frenata e sterzata.

Il livello di comunicazione tra il sistema di Intelligenza Artificiale e TORCS è gestito tramite il protocollo SCR (basato su socket UDP). L'architettura software è stata ingegnerizzata secondo rigidi criteri di modularità, separando logicamente l'interfaccia dell'ambiente di simulazione dai moduli operativi di inferenza. Il campionamento delle variabili di stato opera a una frequenza di circa 50 Hz, un requisito stringente per la gestione delle dinamiche del veicolo ad alte velocità. Per evitare instabilità nel control loop, l'architettura è stata ottimizzata per garantire una latenza di inferenza in tempo reale inferiore ai 10 ms.

Stack Tecnologico e Strumenti:

- **Linguaggio e Framework:** Python 3.x, supportato dalla libreria scikit-learn per lo sviluppo del modello di regressione KNN, e da NumPy e pandas per la manipolazione algebrica e l'elaborazione dei flussi di dati.
- **Assistenza allo Sviluppo (LLM):** Modello IBM Granite (integrato tramite il percorso formativo "IBM SkillsBuild: IBM Granite Models for SW Dev"), impiegato strategicamente per le operazioni di refactoring e per l'ottimizzazione prestazionale del codice sorgente.

Fase di acquisizione

La capacità di generalizzazione del modello è strettamente correlata alla densità e alla varianza del dataset di addestramento. L'acquisizione dei dati è stata effettuata tramite gli script

manual_control_ds4.py e manual_control_ds4_recovery.py, interfacciati con un controller DualShock 4, registrando complessivamente circa 100 giri di pista.

Il set di dati comprende sia traiettorie ottimali (giri veloci) sia manovre di recovery (rientro in pista da posizioni sub-ottimali o perdite di aderenza indotte deliberatamente). L'inclusione di queste ultime è fondamentale per istruire l'agente sulle correzioni di traiettoria, mitigando il fenomeno del covariate shift (l'errore a cascata tipico del Behavioral Cloning, in cui divergenze minime si accumulano portando l'agente verso stati non esplorati nel training set).

Specifiche del Dataset Grezzo (Raw Data):

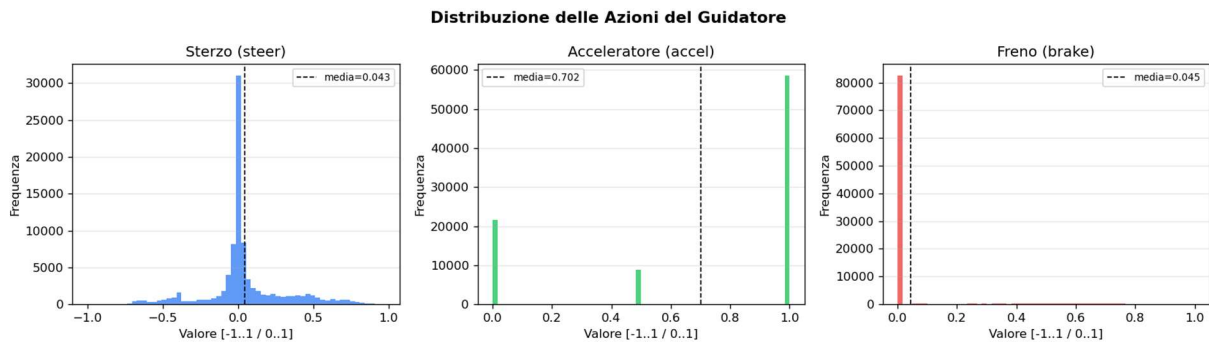
- **Volume totale:** ~100.000 campioni distribuiti su circa 100 giri (salvati in file CSV indipendenti).
- **Feature di stato estratte (30 variabili):** angolo della vettura rispetto all'asse della pista (angle), marcia inserita (gear), regime del motore (rpm), componenti vettoriali della velocità (speedX, speedY, speedZ), posizione trasversale (trackPos), 19 sensori telemetrici di distanza dai bordi della pista e 4 sensori di rotazione delle ruote (wheelSpinVel).
- **Variabili target estratte (4 variabili):** sterzo (target_steer), accelerazione (target_accel), frenata (target_brake) e marcia (target_gear).

Pre-elaborazione dei Dati e Feature Engineering

I dati grezzi acquisiti vengono elaborati tramite la pipeline implementata nello script step1_prepare_data.py. In questa fase, attraverso tecniche di feature selection e feature engineering, lo spazio di input viene ridotto a 24 variabili salienti (angle, trackPos, speedX, speedY, wsv_avg e i 19 sensori telemetrici track), mentre lo spazio di output è limitato a 3 variabili target continue (target_steer, target_accel, target_brake).

L'elaborazione si articola nelle seguenti fasi procedurali:

- **Aggregazione dei Dati:** Fusione dei singoli file CSV grezzi in un unico dataframe consolidato.
- **Filtraggio e Selezione delle Traiettorie:** Estrazione di un sottoinsieme ottimizzato di circa 30 giri, ottenuto bilanciando traiettorie veloci e manovre di recovery. La coerenza dei campioni è valutata in funzione dello spostamento rispetto alla traiettoria mediana. Il set risultante è memorizzato nel file dataset_merged.csv, supportato dalla generazione di plot per l'analisi visiva delle derive di traiettoria e da file di log per la tracciabilità delle fonti.
- **Pulizia e Bilanciamento:** Rimozione dei campioni anomali (outlier) e bilanciamento della distribuzione dei dati. Nello specifico, viene applicato un undersampling ai dati campionati in rettilineo per prevenire bias predittivi derivanti dalla naturale sovrarappresentazione dell'angolo di sterzata nullo (bassa varianza dello sterzo). L'output di questa fase genera il file dataset_clean.csv.
- **Standardizzazione delle Feature:** Applicazione di una trasformazione lineare tramite StandardScaler (implementato in scikit-learn) per ricondurre le feature a una distribuzione con media nulla e varianza unitaria. Gli oggetti di trasformazione e mappatura (scaler.pkl e feature_names.pkl) vengono serializzati per preservare la consistenza distributiva durante la successiva fase di inferenza real-time.
- **Exploratory Data Analysis (EDA):** Generazione automatizzata di grafici analitici per ispezionare il posizionamento spaziale del veicolo e la distribuzione statistica delle variabili di controllo all'interno del training set.

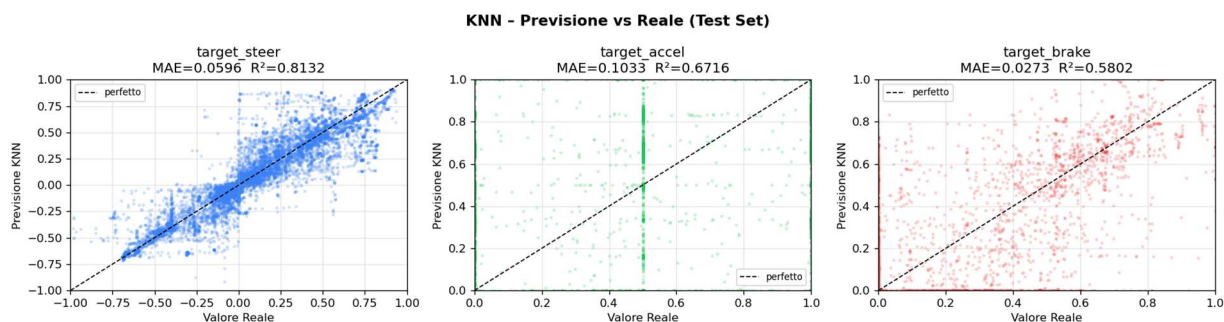


Modellazione tramite K-Nearest Neighbors (KNN)

L'addestramento dell'agente è implementato nello script `step2_train_knn.py`. L'obiettivo di questa fase è lo sviluppo di un modello di regressione multi-output basato sull'algoritmo K-Nearest Neighbors (KNN), progettato per mappare simultaneamente lo spazio degli stati (feature sensoriali) nelle corrispondenti variabili di controllo continue (sterzo, accelerazione, freno).

L'elaborazione si articola nella seguente pipeline:

- **Inizializzazione:** Caricamento del dataset pre-elaborato (`dataset_clean.csv`) e degli artefatti di standardizzazione (`scaler.pkl`, `feature_names.pkl`). I dati vengono separati nella matrice delle feature (X) e nella matrice dei target (y).
- **Splitting e Prevenzione del Data Leakage:** La partizione tra training set (80%) e test set (20%) è gestita tramite la classe `GroupShuffleSplit`. Questo approccio vincola i campioni appartenenti al medesimo giro di pista (lap) a rimanere nello stesso split, prevenendo la contaminazione dei dati in fase di addestramento (data leakage) e garantendo la misurazione di metriche di generalizzazione realistiche.
- **Addestramento (Training):** Configurazione del `KNeighborsRegressor` (tramite `scikit-learn`). Per ottimizzare i tempi di ricerca spaziale è stata adottata la struttura dati `ball_tree` con metrica euclidea. L'iperparametro `weights="distance"` è stato impostato per ponderare il contributo dei vicini in proporzione inversa alla loro distanza. L'esecuzione è vincolata a un singolo thread (`n_jobs=1`) per minimizzare l'overhead di parallelizzazione durante l'inferenza single-query.
- **Valutazione delle Metriche:** Test del modello e calcolo indipendente, per ciascuna variabile target, delle metriche di errore: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) e coefficiente di determinazione (R^2).
- **Analisi Visiva delle Performance:** Generazione di scatter plot (predizioni vs. valori reali) per valutare l'affidabilità del modello ai margini della distribuzione, e di istogrammi per l'analisi dei residui (distribuzione degli errori di predizione).
- **Validazione della Latenza e Serializzazione:** Il modello addestrato viene serializzato (`knn_model.pkl`). Contestualmente, viene eseguito un benchmark simulando 500 inferenze per misurare il tempo di esecuzione medio: una latenza superiore a 10 ms genera un avviso di criticità, essendo il limite architetturale imposto dal control loop del simulatore (che opera a 50 Hz, corrispondenti a una finestra temporale massima di 20 ms per frame).
- **Hyperparameter Tuning:** È integrata una routine opzionale per la ricerca esplorativa dell'iperparametro k (numero di vicini) ottimo, basata sulla minimizzazione dell'errore di validazione.



Implementazione dell'Agente e Risultati

La fase di inferenza in tempo reale è governata dallo script `step3_knn_drive.py`, che implementa il control loop dell'agente autonomo incaricato di pilotare la vettura all'interno dell'ambiente TORCS. L'esecuzione del modulo si articola nelle seguenti routine operative:

- **Inizializzazione e Connessione:** Caricamento in memoria degli artefatti serializzati (il modello `knn_model.pkl`, l'oggetto di standardizzazione e la topologia delle feature) e sincronizzazione della connessione UDP (protocollo SCR) con il server di simulazione.
- **Ciclo di controllo:** Ad ogni tick di simulazione, il sistema acquisisce il vettore delle feature, lo standardizza per garantire coerenza distributiva con il training set ed esegue l'inferenza sul modello KNN per la stima multi-output in tempo reale delle variabili di controllo (sterzo, accelerazione, freno).
- **Sistemi di Assistenza:** Per mitigare le instabilità dinamiche, l'output predittivo viene affinato tramite logiche deterministiche, delegando la trasmissione a un sistema rule-based parametrizzato sulla velocità longitudinale e implementando routine di sicurezza (Traction Control e ABS) per gestire gli slittamenti differenziali e il bloccaggio delle ruote.
- **Attuazione:** Incapsulamento e trasmissione dei comandi operativi definitivi al simulatore tramite datagrammi UDP.

Il **progetto DYNEM** ha validato l'efficacia del Behavioral Cloning nella sintesi di policy di guida robuste. Superate le criticità architetturali legate alla latenza di inferenza e mitigato il *covariate shift* tramite l'integrazione di dati di recovery, l'agente ha dimostrato una solida stabilità di traiettoria. Il sistema ha concluso con successo un giro autonomo sul tracciato stabilendo un tempo di *1:11.023* e registrando una velocità di punta di *254 km/h*.

